Universidad Cooperativa de Colombia

Facultad de Ingeniería · Ingeniería de Software

Estructuras de Datos · ID 706132

Juego de mazmorra: motor 2D con pathfinding

Proyecto integrador · Temática A · Informe técnico final

Andres Agudelo

11 de noviembre de 2026

1. Descripción del sistema

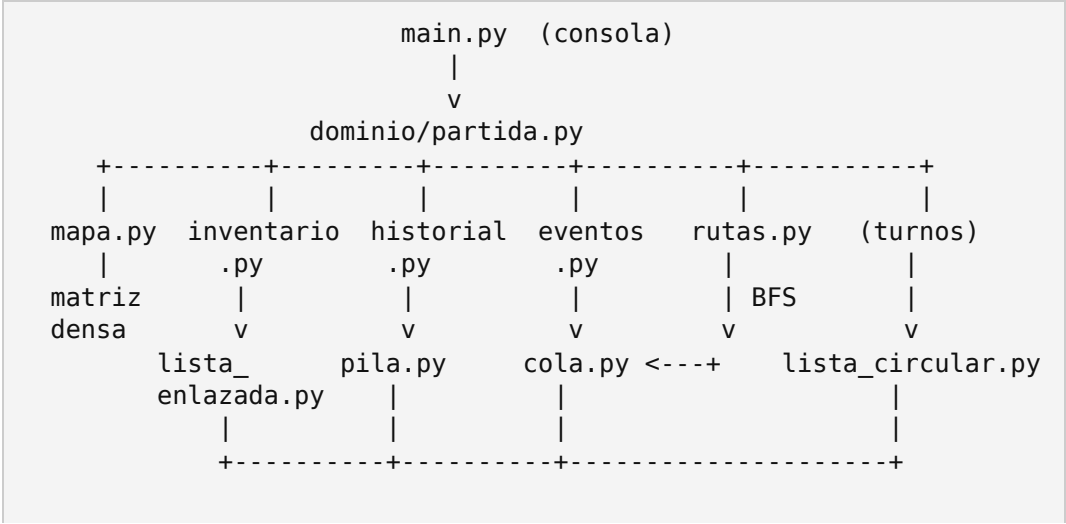
El proyecto es el núcleo lógico de un juego por turnos que se juega en la consola. Un aventurero recorre una mazmorra de 20x20 celdas, con un 15 % de obstáculos puestos al azar, y debe recoger tres tesoros. Dos monstruos lo persiguen: en cada turno calculan el camino más corto hacia el jugador y avanzan una casilla. El jugador gana si recoge todos los tesoros y pierde si un monstruo llega a su celda, salvo que tenga una espada: en ese caso el monstruo muere, reaparece en otra parte del mapa y la espada se gasta.

Además de moverse, el jugador puede recoger, usar y soltar objetos, consultar su inventario, y deshacer o rehacer sus jugadas con todos sus efectos: si deshace la recogida de un objeto, el objeto vuelve al mapa. Lo que pasa en cada turno (aparición de monstruos, recolecciones, colisiones, victoria y derrota) se muestra como eventos en el mismo orden en que ocurrió.

El sistema cubre las siete historias Must del producto (HU-01 a HU-07) y la historia HU-12 (pelear con la espada), con 20 criterios de aceptación verificados por 156 pruebas automáticas.

2. Arquitectura y diagrama de módulos

El código se divide en tres capas. `estructuras/` contiene una estructura de datos por módulo, sin lógica del juego. `dominio/` contiene la lógica del juego y usa esas estructuras. `main.py` es la interfaz de consola: lee comandos, llama a la partida y dibuja el mapa.



```
                                  |
                          estructuras/nodo.py
```

Figura 1. Módulos del sistema y sus dependencias.

Un turno sigue siempre el mismo camino. La consola llama a una acción de la partida. La partida consulta el mapa; si la acción no se puede hacer, por ejemplo por un obstáculo, devuelve el motivo y el turno no pasa. Si se puede, actualiza mapa e inventario, guarda la acción en el historial y anota los eventos. Luego revisa victoria y derrota. Si la partida sigue, la lista circular de turnos le da el turno a cada monstruo, que pide su siguiente paso a las rutas. Al final la consola dibuja el mapa y muestra los eventos.

3. Cómo se aplicó Spec-Driven Development

Cada componente se especificó antes de programarse, en tres archivos dentro de `specs/`: `spec.md` describe qué hace (operaciones, errores, criterios y casos extremos), `plan.md` explica cómo (estructura, complejidad y alternativa descartada) y `tasks.md` lista las tareas y el criterio que cierra cada una. El historial de Git lo muestra: en cada entrega, el commit de specs aparece antes que el de código.

Cada criterio de aceptación tiene un identificador (CA-01 a CA-18, CA-30 y CA-31) y cada prueba dice en su docstring qué criterio verifica. El archivo `specs/trazabilidad.md` permite ir del criterio a la prueba y al código, y también del código al criterio.

Specs que cambiaron y por qué

Archivo	Cambio	Motivo
01-historial/spec.md 1.2	Deshacer sin historial avisa, sin error	La versión anterior decía que lanzaba un error y contradecía a la historia HU-04
02-mapa/spec.md 1.1	Se agrega `objeto_en`	La consola necesita leer cada celda para dibujar el mapa
07-partida/spec.md 1.0	Spec nueva	La partida existía desde la Entrega 2 sin spec propia
moscow.md 1.2	HU-08, HU-09 y HU-10 pasan de Should a	El tiempo alcanzó para los Must completos y probados

	Won't	
estructuras.md 1.2	Salen la tabla hash y el montículo	Solo los usaban las historias que pasaron a Won't
historias.md 1.3 y 07-partida/spec.md 1.1	Nueva HU-12: pelear con la espada	Darle uso a la espada con poco código, sin estructuras nuevas

El cambio más importante fue el último. Al final del proyecto había que elegir entre construir las ocho estructuras previstas o entregar bien probadas las que resuelven los mínimos del dominio. Se eligió lo segundo y la decisión quedó registrada con su motivo, como permite la metodología.

4. Estructuras de datos

4.1 Dos pilas: historial de deshacer y rehacer

Qué es. Una pila guarda elementos de forma que el último en entrar es el primero en salir. Se implementó sobre nodos enlazados, con el tope en la cabeza.

Cómo se implementó. `estructuras/pila.py` (adaptada del laboratorio de la semana 8) y `dominio/historial.py`. El historial tiene dos pilas: `hechas` y `deshechas`. Registrar hace push en hechas y vacía deshechas. Deshacer pasa la acción de hechas a deshechas, y rehacer hace lo contrario.

Dónde se usa. Cada acción del jugador se guarda como un objeto `Accion` con lo necesario para revertirla: posición anterior, posición nueva y objeto recogido o soltado.

Complejidad. Registrar, deshacer y rehacer son O(1).

Alternativa descartada. Lista doble con cursor. También es O(1), pero hay que mantener el cursor y cortar la lista en cada acción nueva.

4.2 Lista enlazada simple: inventario

Qué es. Una cadena de nodos donde cada uno apunta al siguiente. Guarda referencias a la cabeza y a la cola.

Cómo se implementó. `estructuras/lista_enlazada.py`, adaptada del laboratorio de la semana 6. Se completaron los dos métodos que el laboratorio dejaba sin terminar: insertar en medio y eliminar un nodo que no es el primero. `dominio/inventario.py` agrega al inicio y quita buscando el objeto por nombre.

Dónde se usa. En el inventario del jugador y para guardar a los monstruos de la partida. En la pelea, la espada se quita del inventario con la misma operación `quitar`.

Complejidad. Agregar O(1); quitar y listar O(n).

Alternativa descartada. Arreglo dinámico. Buscar por nombre cuesta O(n) en ambas, pero el arreglo además debe correr los elementos al quitar uno del medio, y el juego nunca usa el acceso por posición.

4.3 Cola enlazada: eventos y BFS

Qué es. Una estructura donde el primero en entrar es el primero en salir. Se encola por la cola y se desencola por la cabeza.

Cómo se implementó. `estructuras/cola.py`, la `ColaEnlazada` del laboratorio de la semana 9. `dominio/eventos.py` registra cada evento como texto y los saca todos en orden al final del turno.

Dónde se usa. En los eventos del turno y, como estructura auxiliar, en la búsqueda en anchura de las rutas.

Complejidad. Encolar y desencolar O(1); sacar todos O(n).

Alternativa descartada. Cola de prioridad: insertar cuesta O(log n) y ningún evento es más urgente que otro.

4.4 Lista circular: orden de turnos

Qué es. Una lista enlazada donde el último nodo apunta al primero, así que recorrerla no tiene fin.

Cómo se implementó. `estructuras/lista_circular.py`, adaptada del laboratorio de la semana 7. Guarda una referencia al nodo actual; avanzar es `actual = actual.siguiente`.

Dónde se usa. La partida agrega al jugador y a cada monstruo. Después de que el jugador actúa, se avanza por los monstruos hasta volver al jugador.

Complejidad. Agregar, consultar el actual y avanzar son O(1).

Alternativa descartada. Cola: cada turno exige desencolar y volver a encolar.

4.5 Grafo implícito con BFS: rutas de los monstruos

Qué es. El mapa se ve como un grafo: cada celda sin obstáculo es un vértice y sus vecinos son las celdas libres de arriba, abajo, izquierda y derecha. Es implícito porque no se guarda una lista de aristas; los vecinos se calculan al momento. La búsqueda en anchura (BFS) recorre el grafo por capas de distancia.

Cómo se implementó. `dominio/rutas.py`, función `siguiente_paso`. Encola la celda del monstruo; mientras la cola no esté vacía, desencola una celda y encola sus vecinos no visitados, anotando en una matriz `padre` de dónde vino

cada uno. Al llegar al jugador, camina hacia atrás por los padres hasta encontrar la primera celda del camino.

Dónde se usa. Cada monstruo la llama en su turno. Si no hay camino, el monstruo se queda quieto.

Complejidad. O(V + E). Con V = 400 celdas y como máximo 4 vecinos por celda, queda O(f·c).

Alternativa descartada. DFS cuesta lo mismo pero no garantiza el camino más corto, que es lo que pide CA-13. A* exploraría menos celdas, pero en un mapa de 20x20 la diferencia no se nota y se dejó fuera.

4.6 Matriz densa: mapa

Qué es. Un arreglo de filas, donde cada fila es un arreglo de celdas.

Cómo se implementó. `dominio/mapa.py`. Cada celda guarda `None` si está libre, `"#"` si tiene obstáculo o el nombre del objeto. Las listas de Python se usan como arreglos de tamaño fijo: se crean una vez y no crecen.

Dónde se usa. En cada movimiento del jugador, en BFS, en la revisión de victoria y para dibujar.

Complejidad. Consultar y modificar una celda O(1); crear el mapa O(f·c).

Alternativa descartada. Matriz dispersa. Con un 15 % de obstáculos ahorra poca memoria y la consulta deja de ser O(1).

5. Dificultades

La primera dificultad fue que deshacer y rehacer revirtieran también los efectos de cada acción. Se resolvió guardando en cada acción lo necesario para revertirla, en lugar de guardar solo su nombre.

La segunda fue entender cómo BFS reconstruye el camino. BFS no guarda caminos completos: solo anota, para cada celda, desde qué celda se llegó. El camino se recupera caminando hacia atrás desde el destino.

La tercera fue el tiempo. Al llegar a la entrega final quedaban pocos días y había que decidir entre construir todas las historias o terminar bien las obligatorias. Se priorizaron los Must y los Should pasaron a Won't con su motivo registrado.

6. Declaración de uso de inteligencia artificial

Se usó un asistente de IA (Claude) para organizar los artefactos SDD y la priorización MoSCoW, revisar las specs contra la rúbrica, simplificarlas, planear los commits, escribir parte del código (la partida, la consola, BFS, el fin de partida) y las pruebas, completar los métodos faltantes de la lista enlazada de Clase y preparar un borrador de este informe. Las estructuras base (nodo, pila, lista enlazada, lista circular y cola) parten de los laboratorios del curso. Cada commit fue revisado antes de aprobarlo, y el uso de IA también se declara en `BITACORA.md`.

7. Conclusiones

La estructura correcta depende de la operación que más se repite. El historial usa pilas porque siempre opera sobre lo más reciente; los eventos usan una cola porque importa el orden de llegada; los turnos usan una lista circular porque no tienen fin; el mapa usa una matriz porque se consulta por coordenadas.

Una misma estructura puede servir para más de un problema. La cola de los eventos es la misma que usa BFS, y la lista enlazada guarda tanto el inventario como a los monstruos.

Escribir la spec antes del código hizo visibles las preguntas difíciles a tiempo, como qué pasa al deshacer con el historial vacío. Y registrar los cambios con su motivo, incluido el recorte de alcance final, dejó claro qué se decidió y por qué.

8. Bibliografía

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data structures and algorithms in Python*. Wiley.

Python Software Foundation. (2024). *The Python tutorial*. https://docs.python.org/3/tutorial/

Universidad Cooperativa de Colombia. (2026). *Proyecto integrador, temática A: Motor de juego 2D con pathfinding* [Documento técnico del curso Estructuras de Datos].